

Google Play Games Services Setup

Step-by-step for enabling sign-in, achievements, leaderboards, and cloud save on Android. Used by the `ProfileBridge` implementation in `CapacitorPlatformBridge`.

What this gets us

- **Sign-in**: the player's Play Games profile (display name, avatar, stable player ID). Shown in-app instead of the web anonymous profile.
- **Achievements**: unlock events surfaced via the native Play Games UI + a nice icon shelf on the player's profile.
- **Leaderboards**: per-board submitted scores visible to the player + friends.
- **Cloud save**: snapshot API for persisting JSON state across devices tied to the Play Games account.

iOS has a parallel system (**Game Center**) with a similar feature set. Setup for Game Center is a separate doc; the `ProfileBridge` contract is platform-agnostic so feature code doesn't branch.

Prerequisites

- Google Play Console account (\$25 one-time fee, same one you'll use to publish the Play Store listing).
- The app's **package name** registered: `com.alexhughes.factions`.
- A **release signing key** (or debug keystore fingerprint for testing) — Play Games Services binds auth to the signing certificate.

Verify your fingerprint:

```
# Debug keystore (auto-generated by Android Studio the first time you run the app)
keytool -list -v -keystore ~/.android/debug.keystore -alias androiddebugkey -storepass android

# Production keystore (once you mint it)
keytool -list -v -keystore /path/to/release.keystore -alias <your-alias>
```



Copy the **SHA1** line — you'll paste it into Play Console.

1. Create a game project in Play Console

1. [Play Console](#) → **Grow users** → **Play Games Services** → **Setup and management** → **Configuration**.
 2. Click **Add new Play Games Services project**.
 3. Pick one of:
 - **Yes, my game already uses Google APIs** — link to an existing Google Cloud project.
 - **No, my game doesn't use Google APIs** — creates a new Cloud project automatically. For a fresh setup, pick "No". Play Console provisions the project.
 4. Name the project (internal only): `Factions – Play Games`.
 5. Properties:
 - Display name: `Factions`.
 - Description: short marketing copy.
 - Category: `Games`.
 - Developer name: your developer name.
-

2. Configure credentials (OAuth)

1. In the Play Games Services section → **Credentials** → **Add credential**.
2. Type: **Android**.
3. Package name: `com.alexhughes.factions`.
4. SHA-1 fingerprint: paste the one from the keytool output above.
5. Grant **Drive API** scopes if you want cloud save (we do).
6. Complete the OAuth consent screen if prompted — set the same app name, support email, and developer URL you used on the Play Store listing.

Repeat this step for each signing certificate you want to authorise:

- The debug keystore on each dev machine.
- The production release keystore.

3. Mint achievements

For each achievement you want:

1. Play Games Services → **Achievements** → **Add new achievement**.
2. Fill in:
 - **Name** (shown in the UI).
 - **Description**.
 - **Icon** (512×512 PNG, Google's content guidelines apply).
 - **Points**: your choice; total across all achievements should feel meaningful but isn't strictly bounded.
 - **Incremental** vs **Standard**: incremental for "kill 1000 creeps" progress; standard for "win a match".
3. Save. Copy the **Achievement ID** — an alphanumeric string like `CgkI...`.

Put the IDs somewhere in the codebase — suggested path is `src/data/Achievements.ts` with a typed constant map:

```
export const ACHIEVEMENTS = {
  FIRST_WIN: 'CgkI...',
  TUTORIAL_COMPLETE: 'CgkI...',
  // ... etc
} as const;
```

Call via `platformBridge().profile.unlockAchievement(ACHIEVEMENTS.FIRST_WIN)`.

4. Mint leaderboards

Same flow under **Leaderboards** → **Add new leaderboard**.

- **Name** (e.g. "Highest wave reached — Standard mode").
- **Score format**: Numeric vs Time. Use Numeric for wave counts, Time for speedruns.
- **Ordering**: Larger is better (default for score), Smaller is better (speedrun).
- **Time span**: All-time, weekly, daily. Most boards want all of the above on automatically.

Copy each **Leaderboard ID**. Add to a typed map:

```
export const LEADERBOARDS = {
  STANDARD_WAVE: 'CgkI...',
  ENDLESS_WAVE: 'CgkI...',
  GAUNTLET_TIME: 'CgkI...',
} as const;
```

Submit via `platformBridge().profile.submitLeaderboard(LEADERBOARDS.STANDARD_WAVE, waveNum)` .

5. Enable cloud save

1. Play Games Services → **Saved Games** → **Enable saved games**.
2. Pick an **icon** (shown in the Google Play Games app's save-picker UI).
3. Cover image (optional — shown in the save picker).

The `ProfileBridge.cloudSave()` / `cloudLoad()` methods wrap the Play Games Snapshots API — one named slot per call, JSON payload as a string (we serialise `StoreState` / tutorial state / etc on our side).

Conflict resolution: Play Games auto-resolves by most-recent-write unless you override. For our use (single-player progress), most-recent is fine.

6. Testers + staged rollout

Before publishing the Play Games Services config live, add yourself as a tester:

1. Play Games Services → **Testers** → **Tester email addresses**.
2. Add your Google account + anyone else who needs to test before release.

Until you **publish** the Play Games project, only these testers can sign in. Publish happens under **Publishing** → **Publish my Play Games Services project** . Do this *once Achievements / Leaderboards / Saved Games are all finalised* — edits after publish take effect immediately for all users.

You can unpublish + republish later to iterate on descriptions without losing player data.

7. Install the Capacitor plugin

Once the Play Console side is set up, install the plugin and wire it into the bridge:

```
npm install @capacitor-community/play-games
# (plugin name may differ – see the latest docs; some teams use a
# custom-built plugin wrapping the Android Play Games Services SDK
# directly)
```

Then in `CapacitorPlatformBridge` :

```
// Pseudocode – adapt to the chosen plugin
import { PlayGames } from '@capacitor-community/play-games';

async signIn(): Promise<PlayerProfile | null> {
  const { player } = await PlayGames.signIn();
  return { id: player.playerId, displayName: player.displayName, ... };
}

async unlockAchievement(id: string): Promise<void> {
  await PlayGames.unlockAchievement({ id });
}

async submitLeaderboard(id: string, score: number): Promise<void> {
  await PlayGames.submitScore({ leaderboardId: id, score });
}

async cloudSave(slot: string, json: string): Promise<void> {
  await PlayGames.saveSnapshot({ name: slot, data: json });
}

async cloudLoad(slot: string): Promise<string | null> {
  const { data } = await PlayGames.loadSnapshot({ name: slot });
  return data ?? null;
}
```

8. Testing the integration

Before publishing the Play Games project:

1. Build a debug APK (signed with the debug keystore fingerprint you registered).
2. Install on a test device logged in with a Google account that's in the **Testers** list.
3. Launch the app, trigger sign-in. First time prompts for Play Games account selection; subsequent launches auto-sign-in.
4. Trigger an achievement — it should pop a native toast.
5. Open the Play Games app → Profile → Achievements — verify the unlock appears.

Links

- [Play Games Services overview](#)
- [Enable Saved Games](#)
- [Achievements API](#)
- [Leaderboards API](#)
- [@capacitor-community/play-games](#) (verify this is the current recommendation at integration time)